LAB 10

Software Metrics & Complexity Analysis

Health Insurance Claim Portal

Name	Gagan Sharma
Roll No	202401100200140
Branch / Section	CSE B
Experiment	Lab 10 - Software Metrics & Complexity Analysis

1. Aim and Introduction

Aim: To measure the size, complexity and quality of the Health Insurance Claim Portal using standard software metrics, and to interpret the results so that risky parts of the design can be identified and improved.

Software metrics give quantitative evidence about a product and its process. They support estimation (how big and how costly), quality control (how complex and how maintainable) and decision making (where to refactor and where to test more). In this lab the claim-processing module of the portal (designed in Lab 9) is analysed with the following metrics:

- **Size metrics:** Lines of Code (LOC) and Function Points (FP).
- **Complexity metrics:** McCabe's Cyclomatic Complexity and Halstead measures.
- **Object-oriented metrics:** Chidamber and Kemerer (CK) suite - WMC, DIT, NOC, CBO, RFC, LCOM.
- **Maintainability and estimation:** Maintainability Index and Basic COCOMO.

2. Module Under Analysis

The core function of the portal is *process_claim()*. It checks policy validity, claim amount, duplicates, network-hospital rules, document verification and risk before deciding whether a claim is rejected, flagged, queried, sent for manual review or approved. The listing below is the version analysed.

```
def process_claim(claim, policy):
    if not policy.is_active():
        return "REJECTED"
    if claim.amount <= 0:
        return "REJECTED"
    if claim.amount > policy.sum_insured:
        return "REJECTED"
    if claim.is_duplicate():
        return "FLAGGED"
    payable = claim.amount
    if claim.type == "CASHLESS" and not claim.hospital.is_network():
        payable = payable * 0.8
    for doc in claim.documents:
        if not doc.verified:
            return "QUERY"
    if payable > 50000 or claim.is_high_risk():
        return "MANUAL_REVIEW"
    return "APPROVED"
```

3. Size Metrics

3.1 Lines of Code (LOC)

Measure	Value
Physical lines (non-blank)	18
Comment lines	0
Executable statements (excluding def line)	17
Comment ratio	0%

The function is short, but it has no comments. Adding documentation of each business rule would raise maintainability without changing complexity.

3.2 Function Point Analysis (whole portal)

Function Points measure functionality from the user's viewpoint and do not depend on the programming language. Average complexity weights are used.

Component	Items identified	Count	Weight	Total
External Inputs (EI)	Register, login, submit claim, upload document, update status, make payment	6	4	24
External Outputs (EO)	Claim statement, reports, notifications, payment receipt	4	5	20
External Inquiries (EQ)	View policy, track claim, view document, search claim, claim history	5	4	20
Internal Logical Files (ILF)	Users, policies, claims, documents, payments	5	10	50
External Interface Files (EIF)	Payment gateway, hospital system	2	7	14
Unadjusted FP (UFP)				**128**

The 14 General System Characteristics (data communication, performance, security, online update, etc.) were rated 0-5 and sum to 42. Value Adjustment Factor: VAF = 0.65 + 0.01 x 42 = **1.07**. Adjusted size: FP = UFP x VAF = 128 x 1.07 = **137.0 function points**.

4. McCabe's Cyclomatic Complexity

Cyclomatic complexity V(G) measures the number of linearly independent paths through a program and therefore the minimum number of test cases for full branch coverage. It is calculated as **V(G) = P + 1**, where P is the number of predicate (decision) conditions; equivalently V(G) = E - N + 2 on the control-flow graph.

Decision construct	Count	Where
if statements	7	policy active, amount <= 0, amount > sum insured, duplicate, cashless/network, unverified document, payable/risk
Loop (for)	1	iterating over claim documents
Logical AND	1	cashless *and* not network hospital
Logical OR	1	payable > 50000 *or* high risk
Total predicates P	**10**	

V(G) = 10 + 1 = 11. According to the SEI risk table (1-10 simple, 11-20 moderate, 21-50 high, above 50 untestable), the function lies in the **moderate risk** band, at the edge of the commonly accepted limit of 10.

4.1 Independent (basis) paths

Path	Condition sequence	Outcome
1	Policy inactive	REJECTED
2	Active, amount <= 0	REJECTED

Path	Condition sequence	Outcome
3	Amount > sum insured	REJECTED
4	Valid claim, duplicate	FLAGGED
5	Cashless at non-network hospital (80% rule), all documents verified, low value	APPROVED
6	Claim type not cashless	APPROVED
7	Cashless at a network hospital	APPROVED
8	Document not verified	QUERY
9	No documents (loop skipped)	APPROVED
10	Payable > 50000	MANUAL_REVIEW
11	Payable <= 50000 but high-risk claim	MANUAL_REVIEW

At least **11 test cases** are required, one for each independent path.

5. Halstead Complexity Measures

Halstead's measures are derived from the number of distinct and total operators and operands in the source code.

Symbol	Meaning	Value
n1	Distinct operators (if, not, return, <=, >, ==, =, *, and, or, for, call)	12
N1	Total operators	31
n2	Distinct operands (variables, attributes, constants, strings)	20
N2	Total operands	27

Measure	Formula	Result
Vocabulary n	n1 + n2	32
Length N	N1 + N2	58
Volume V	N x log2(n)	290.0
Difficulty D	(n1 / 2) x (N2 / n2)	8.10
Effort E	D x V	2349
Time T	E / 18 seconds	131 s (about 2.2 min)
Estimated bugs B	V / 3000	0.097

The volume is modest, but difficulty above 8 indicates that many different attributes and constants must be understood together, which agrees with the moderate cyclomatic complexity.

6. Maintainability Index

The Maintainability Index combines Halstead volume, cyclomatic complexity and LOC: **MI = 171 - 5.2 ln(V) - 0.23 V(G) - 16.2 ln(LOC)**.

MI = 171 - 5.2 ln(290.0) - 0.23 x 11 - 16.2 ln(18) = **92.2**. On the original scale, values above 85 mean good maintainability, 65-85 moderate and below 65 difficult. The function therefore falls in the **good**

range, mainly because it is short. The index does not penalise the 11 decision paths strongly, so cyclomatic complexity (Section 4) is still the better warning sign here, and the refactoring in Section 9 is worthwhile. (Normalised to 0-100 as in some tools, the score is 53.9.)

7. Object-Oriented (CK) Metrics

The CK metrics were estimated for the main classes from the class design of Lab 9. WMC is the number of methods weighted by complexity, DIT the depth of inheritance tree, NOC the number of children, CBO the number of coupled classes, RFC the response set size and LCOM the lack of cohesion in methods.

Class	WMC	DIT	NOC	CBO	RFC	LCOM
User	8	0	2	3	12	Low
Policyholder	6	1	0	4	11	Low
ClaimOfficer	7	1	0	5	14	Low
Policy	7	0	0	3	10	Low
Claim	18	0	0	7	26	Medium
Document	5	0	0	2	8	Low
Payment	9	0	0	4	15	Low
Notification	4	0	0	2	7	Low

7.1 Interpretation

- **Claim** has the highest WMC (18), CBO (7) and RFC (26): it is the most complex and most coupled class, a candidate for splitting (for example, moving validation and payable-amount calculation into separate classes).
- DIT is at most 1 and NOC at most 2, so inheritance is shallow, which keeps the design easy to understand and avoids fragile hierarchies.
- CBO values stay at 7 or below and LCOM is low for nearly all classes, which indicates good cohesion and acceptable coupling.
- Classes with high WMC and RFC should receive the largest share of unit tests.

8. Project Estimation (Basic COCOMO)

The portal size is converted to lines of code using an assumed 53 LOC per function point for Java: 137.0 FP x 53 = 7259 LOC = **7.26 KLOC**. The project is treated as *organic* mode (small team, familiar domain), with a = 2.4, b = 1.05, c = 2.5, d = 0.38.

Quantity	Formula	Result
Effort E	2.4 x (KLOC)^1.05	19.2 person-months
Development time D	2.5 x (E)^0.38	7.7 months
Average staffing	E / D	2.5 persons

These are early estimates and should be recalibrated with actual productivity data from the team.

9. Refactoring to Reduce Complexity

To bring every function below the recommended limit of 10 (and improve testability), *process_claim()* is split into small single-purpose functions that follow the Single Responsibility Principle. The Strategy pattern from Lab 9 can further replace the amount rules.

Function after refactoring	Decisions	V(G)
validate_basic(claim, policy): policy active, amount > 0, within sum insured	3 if	4
check_documents(claim): loop and verification test	1 for + 1 if	3
calculate_payable(claim): cashless and non-network rule	1 if + 1 and	3
decide_outcome(payable, claim): high value or high risk	1 if + 1 or	3
process_claim(): orchestrates the above and checks duplicate	3 if	4

The largest V(G) falls from **11 to 4**. The total across functions rises slightly (17), which is normal because the checks are separated, but each function now needs only 3-4 tests, is easier to read, and can be reused and modified independently.

10. Summary of Results

Metric	Result	Interpretation
LOC (process_claim)	18	Small, but undocumented
Function Points	137.0	Medium-sized portal
Cyclomatic Complexity	11	Moderate risk; 11 test paths needed
Halstead Volume / Difficulty	290 / 8.1	Moderate mental effort
Maintainability Index	92.2	Good (above 85) due to small size
Most complex class	Claim (WMC 18, CBO 7)	Candidate for splitting
COCOMO effort / time	19.2 PM / 7.7 months	Organic mode estimate

11. Conclusion

The metric analysis shows that the Health Insurance Claim Portal is a medium-sized system whose design is generally cohesive, with shallow inheritance and manageable coupling. The claim-processing function sits in the moderate-risk band for cyclomatic complexity, and the Claim class concentrates most of the complexity. Refactoring the function into smaller units reduces the maximum complexity from 11 to 4, simplifies testing and improves maintainability. Regular measurement of these metrics during development helps keep complexity under control, supports accurate estimation and guides testing effort to the riskiest modules.

References

- McCabe, T. J., *A Complexity Measure*, IEEE Transactions on Software Engineering, 1976.
- Halstead, M., *Elements of Software Science*, Elsevier, 1977.
- Chidamber, S. and Kemerer, C., *A Metrics Suite for Object Oriented Design*, IEEE TSE, 1994.
- Pressman, R., *Software Engineering: A Practitioner's Approach*, McGraw-Hill.